Name - Varshan Maji

Roll - 230103025

sec-B

Course: Cloud computing

Architecture Overview & pipeline Design.

The core objective: Building a distributed, automated distillation & inference pipeline for a command-parsing small language model (SLM) running locally & in a kubernetes environment.

When I first started this project, the goal seemed simple: build an automated, containerized pipeline to distill a teacher model into a small, efficient command parser, and deploy it onto a local kubernetes cluster. But actually building it gave me a whole new perspective on systems & cloud engineering

I started with data generation using a teacher model - phi 3.5 running locally via Ollama. Instead of processing everything sequentially, I set up parallel worker scripts (worker.py) using environment variables (SHARD_INDEX & SHARD_COUNT) to slice the command pool. These workers queried the model, uploaded the resulting JSON files (shard-0.jsonl & shard-1.jsonl) straight into SeaweedFS, which acted as our local S3-compatible object storage layer at http://localhost:8333.

This part of the architecture really opened my eyes. It mirrors how enterprise production teams handle heavy data ingestion pipelines in AWS or GCP, but entirely on a local development machine. Using SeaweedFS as an intermediate storage boundary decoupled data generation from the cleaning phase. If a worker failed, we didn't lose our progress. It made the entire architecture modular, fault-tolerant & resilient.

Ollama (Teacher) → Parallel workers → SeaweedFS shards → cleaner → Trainer → FastAPI App (+3d).

Debugging Trenches - Data Cleaning & Environment Friction :

It turned that when i ran the cleaner, my stats showed kept:0 - every single record was getting rejected under reason code - a 'JSON parsing failure'.

It turned out that while phi3.5 is a solid model, it loves being conversational & frequently wrapped it's JSON o/p in markdown code blocks.

Beyond data issues, I ran into classic local environment friction. Working on macOs with ARM64 architecture meant dealing with PyTorch multiprocessing quirks & missing library dependency like accelerate.

Containerization, kubernetes, & locust load Testing:

The final stretch of the project shifted from data pipelines to infrastructure & observability. Once the model weights were trained, saved & scored using the validation set (score.py), it was time to package everything into a FastAPI microservice, build the Docker container (command-parser:v1), and deploy it into a local K3d kubernetes cluster.

Setting up the kubernetes manifests (k8s/deployment.yaml) with isolated namespaces (data, pipeline, app) made me realize why orchestration tools are non-negotiable in modern software development. Manging container replicas, health checks, & local image imports (k3d image report) felt like running a real mini-production cloud environment.

To poove the deployment could handle real traffic, we executed the Locust load test (locustfile.py) hammering our ingress endpoint at http://localhost:8080 with concurrent users over a 2-minute span. Analyzing the resulting CSV metrics in results/ was deeply satisfying. I could directly see how CPU limits & replica scaling affected our P95 latency & throughput.

Summary :

① key learnings :

(i) Containerization & Orchestration.

(ii) API & Model Integration

(iii) Kubernetes Networking

② Challenges faced :

(i) Resource constraints

(ii) Inter-Service Communication & Configuration

(iii) Container Image Footprint

③ Solutions Implemented

(i) Resource Limiting & Optimization

(ii) Kubernetes Abstractions.

(iii) Optimized Image Builds.

④ key Findings :

(i) Simulation Efficiency

(ii) Asynchronous Performance.

⑤ Proposed Improvements :

(i) CI/CD pipeline integration

(ii) Autoscaling

(iii) Observability

Link: https://github.com/Varshon0nly0code/23010302S-cloud-computing-assignment-7th-sem

zero